```
from datasets import load_dataset

ds = load_dataset ( "Q-b1t/IMDB-Dataset                    Backup " )
```

```
/usr/10ca1/1ib/python3.12/dist-packages/huggingface_hub/uti1s/_auth.py:94: UserWarning:
The secret          TOKEN* does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings . warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher r
WARNING: huggingface_hub.uti1s._http:Warning: You are sending unauthenticated requests to the HF Hub. Plea
README-.md: 100% 23.0/23.0 2.41kB/s] archive.zip: 100% 27.0M/27.OM 135MB/s]

Generating train split: 100%                    50000/50000                52909.03 examples/s]
```

```
from datasets import load_dataset

# Load IMDB dataset dataset =
load_dataset ( " imdb " )

# Check dataset structure print(dataset)
```

```
README-.md:       7.81k/?                822kB/s]
plain_text/train-00000-of00001                    21.0M/21.OM                105MB/s]
.parquet: 100%

                                                  20.5M/20.5M                102MB/s]
plain_text/test-00000-of00001
.parquet: 100%

plain_text/unsupervised-00000-of-                 42.0M/42.OM                68.5MB/s]
00001.p(...): 100%
Generating train split: 100%                    25000/25000                120015.11 examples/s]

Generating test split: 100%                     25000/25000                120239.20 examples/s]

Generating unsupervised split: 100%             50000/50000                183137.41 examples/s]
DatasetDict({    train:    Dataset({
    features: [ ' text ' , ' label num
    rows: 25000

    test:  Dataset({  features  :  [  '
        text ' , ' label num rows:
        25000

     ns oervised: Datas e
```

```
from transformers import DistilBertTokenizerFast
tokenizer = DistilBertTokenizerFast .                "distilbert-base-uncased" )

def tokenize(batch) :
    return                "text"],                , truncation=True)

tokenized dataset = dataset. map(tokenize, batched-True)
```

```
tokenizer_config.json: 100%                                    48.0/48.0          2.84kB/s]
vocab.txt:        232k/?          1.05MB/s]
tokenizer.json:        466k]?          1.26MB/s]
Map: 100%                                        25000/25000        1319.89 examples/s]
Map: 100%                                        25000/25000        1542.96 examples/s]
Map: 100%                                        50000/50000        1484.83 examples/s]
```

```
train dataset =                "train"]
test dataset =                "test"]
```

```
import torch
class SentimentDataset(torch.uti1s.data.Dataset) :
    def init_ (self, encodings, labels) :
        self . encodings = encodings self .
        labels = labels
    def                return
        len(self.labels)
    def _getitem_(self, idx)
    :
        item = {key: torch. tensor(val[idx] ) for key, val in self. encodings.
        items()} item [ " labels " ] = torch. tensor(self.labels[idx] ) return item
```

```
# Extract encodings and labels from the tokenized_ dataset
train_encodings input_ ids' : tokenized_dataset [ "train" ] ' input_ ids ' ] ,
attention mask' : tokenized_dataset [ "train " ] [ ' attention_mask ' ]
train labels = tokenized_dataset [ "train" ] [ ' label ' ]
test_encodings input_ ids' : tokenized_dataset [ "test" ] [ ' input_ ids ' ] ,
    attention mask' : tokenized_dataset[ "test " ] [ ' attention_mask ' ]
test labels = tokenized_dataset [ "test" ] [ ' label ' ]
train dataset = SentimentDataset(train_encodings, train_ labels) test dataset -
SentimentDataset(test_encodings, test _labels)
```

```
from transformers import pipeline
# Load pretrained sentiment analysis pipeline (fast, no training)
sentiment_pipeline - pipeline( " sentiment-analysis " )
# Test on a few sample sentences texts
    "I loved this movie, it was fantastic !
    "The film was boring and too long. '
    "An average experience, not too bad but not great. "
results = sentiment_ pipeline
(texts) for text, res in zip(texts,
results) :
    print (f" Text: {text}\nPrediction: {res}\n")
```

No model was supplied, defaulted to distilbert/disti1bert-base-uncased-finetuned-sst-2-eng1ish and revisi(
Using a pipeline without specifying a model name and revision in production is not recommended.
config.json: 100% 629/629 68.9kB/s]

model.safetensors: 100% 268M/268M 54.7MB/s]

Loading weights: 100% 104/104 440.21 it/s, Materializing param=pre_classifier.weight]

tokenizer_config.json: 100% 48.0/48.0 4.78kB/s]

vocab.txt: 232k/? 12.2MB/s]
Text: I loved this movie, it was fantastic!
Prediction: { ' label 'POSITIVE ' , ' score 'o .9998804330825806}
Text: The film was boring and too long.

Prediction: label' : 'NEGATIVE ' , ' score ' o . 9997709393501282}
Text: An average experience, not too bad but not great. Prediction: label
'NEGATIVE ' , ' score ' o .9976065158843994}

```
from sklearn . feature_extraction . text import CountVectorizer
from sklearn. naive_bayes import MultinomialNB from
sklearn.metrics import accuracy_score
```

```
# Tiny subset for speed train texts = dataset
[ "train " ] [ "text " ] [ : 1000] train labels =
dataset [ "train " ] [ " label " ] [ : 1000] test
texts = dataset [ "test " ] [ "text" ] [ : 500] test
labels = dataset [ "test " ] [ " label " ] [ : 500]
```

```
vectorizer = CountVectorizer( " english " , max_features=2€ee) X train -
vectorizer . fit_transform (train_texts) X test = vectorizer . transform
(test_texts)
nb = MultinomialNB()
nb.fit(X_train, train_ labels)
preds = nb . predict (X_test)
```

```
print( "Naive Bayes Accuracy: " accuracy_score(test_labels, preds))
```

Naive Bayes Accuracy: 1.0

```
import torch. nn as nn

class CNNMode1(nn.Modu1e) :
    def init_ (self, vocab_size, embed_dim, num_ classes) :
        super(CNNMode1, self) . init_() self . embedding -
        nn.Embedding(vocab_size, embed_dim) self . convl - nn . 100,
        kernel_size=3) self . pool = nn.MaxP0011d(2) self.fc = nn.
        Linear(100, num_classes)

    def forward(self, x) : self .
            embedding(x)
            x. permute(0,
        2, 1) self . convl (x)
        self . pool (x) x =
        torch. relu(x) x = x.
```

mean(dim=2)
return self.fc(x)

model cnn = CNNMode1 embed dim=50, num_c1asses=2) print (model_cnn)

```
CNNModel(
  (embedding): Embedding(2000, 50)
  (conv1): Conv1d(50, 100, kernel_size=(3,), stride=(1,))
  (pool): MaxPool1d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc): Linear(in_features=100, out_features=2, bias=True)
)
```